CRM DE ATENDIMENTO E ORÇAMENTOS

Entendendo o Backend

Parte 4 — API REST

Abrindo o sistema para o frontend: rotas HTTP, JSON e erros padronizados

1 2 3 4 5

Parte 4 de 5 do backend

VISÃO GERAL

Onde estamos no plano do backend

1

Fundação + Domínio

Regras do negócio em Python puro, com testes.

✓ CONCLUÍDA

2

Casos de uso

As ações do usuário, orquestrando o domínio.

✓ CONCLUÍDA

3

Persistência

Banco de dados, tabelas e migrations.

✓ CONCLUÍDA

4

API REST

As rotas HTTP que o frontend chama.

VOCÊ ESTÁ AQUI

5

Autenticação

Login com JWT e gestão de usuários.

A API é a porta de entrada do backend: o painel Kanban (frontend) conversa com o sistema só por ela, trocando mensagens em JSON.

De novo, nada abaixo mudou: domínio, casos de uso e repositórios são os mesmos. A API só traduz HTTP para chamadas de caso de uso.

CONCEITO

Uma conversa HTTP: pedido e resposta

Pedido (o frontend envia)

```
PATCH /api/v1/orcamentos/ed2c.../estagio
Content-Type: application/json

{ "estagio": "EM_NEGOCIACAO" }
```



Resposta (a API devolve)

```
HTTP 200
{ "id": "ed2c...",
  "estagio": "EM_NEGOCIACAO",
  "valor": "4200.00", ... }
```

1 Método

O verbo: GET lê, POST cria, PATCH altera uma parte.

2 Rota

O endereço do recurso: /orcamentos/{id}/estagio.

3 Status

200 ok · 201 criado · 404 não existe · 422 a regra não deixa.

REST é só uma convenção: cada “coisa” do sistema (cliente, orçamento, interação) tem um endereço, e o método diz o que fazer com ela.

O QUE FOI CRIADO

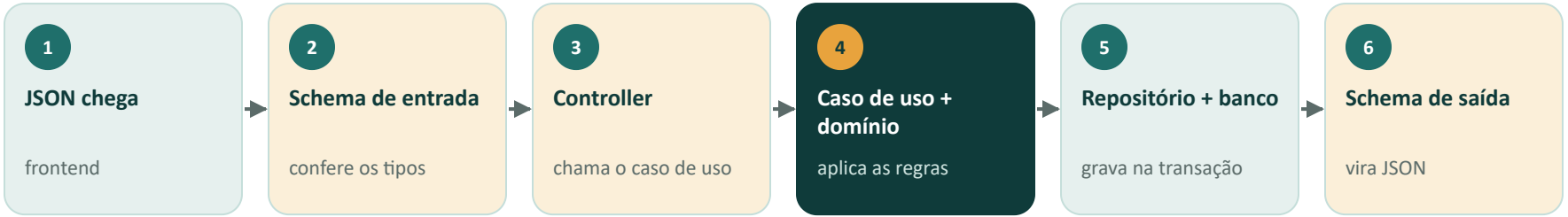
As 9 rotas da API (/api/v1)

POST	/clientes	UC01 · registrar novo contato	201 · 422
GET	/clientes/{id}	UC09 · cliente com orçamentos	200 · 404
GET	/clientes/{id}/historico	UC09 · linha do tempo	200 · 404
GET	/orcamentos	UC07 · painel Kanban	200
GET	/orcamentos/transicoes	apoio ao Kanban	200
PATCH	/orcamentos/{id}/estagio	UC03 · mover estágio	200 · 404 · 422
PATCH	/orcamentos/{id}/valor	UC05 · definir valor	200 · 404 · 422
POST	/orcamentos/{id}/interacoes	UC06 · registrar interação	201 · 404 · 422
GET	/metricas/funil	UC08 · métricas	200

A documentação interativa (Swagger) é gerada sozinha pelo FastAPI em /docs.

O DESENHO

O caminho de uma requisição pelas camadas



Onde cada etapa mora:

Etapas 2 e 6	`adapters/schemas/`	traduzem JSON ↔ objetos Python, em camelCase
Etapa 3	`adapters/controllers/`	uma função por rota, sem nenhuma regra de negócio
Etapa 4	`application/ + domain/`	as mesmas classes das Partes 1 e 2
Etapa 5	`adapters/repositories/`	o repositório Postgres da Parte 3

Se der erro em qualquer etapa, ele vira uma resposta 422 ou 404 no formato padrão — e a transação do banco é desfeita.

Schemas: o contrato do JSON

```python
class Esquema(BaseModel):
    model_config = ConfigDict(
        alias_generator=to_camel,   # snake_case -> camelCase
        populate_by_name=True,
    )

class NovoContatoEntrada(Esquema):
    nome: str
    telefone: str
    email: str
    empresa: str | None = None
    tipo_embalagem_interesse: str | None = None
```

No Python (snake_case)

```
tipo_embalagem_interesse
```

No JSON (camelCase)

```
"tipoEmbalagemInteresse"
```

Tipo × regra

O schema só confere o tipo (telefone é texto). Se o telefone é válido, quem decide é o domínio — a regra não é duplicada.

Cada schema de saída tem um método de(...) que monta o JSON a partir da entidade: OrcamentoSaida.de(orcamento). Assim a entidade nunca vira JSON direto.

ETAPA 3

Controller: uma função fina por rota

```python
@router.patch("/{orcamento_id}/estagio")
def mover_estagio(
    orcamento_id: UUID,             # vem da URL
    entrada: MoverEstagioEntrada,   # vem do corpo JSON
    orcamentos: Orcamentos,         # injetado pelo FastAPI
    relogio: RelogioDaApp,          # injetado pelo FastAPI
) -> OrcamentoSaida:
    orcamento = MoverEstagioUseCase(orcamentos, relogio).executar(
        orcamento_id, entrada.estagio
    )
    return OrcamentoSaida.de(orcamento)
```

1 **Rota**

@router.patch liga a função ao método e ao endereço.

2 **Parâmetros**

O FastAPI preenche cada um: da URL, do corpo ou de uma dependência.

3 **Sem regra**

Não há nenhum if sobre estágio: quem decide é o Orcamento.

Se o estágio enviado não existe (ex.: “QUALQUER”), o FastAPI recusa antes mesmo de chamar a função — e devolve 422 DadosInvalidos.

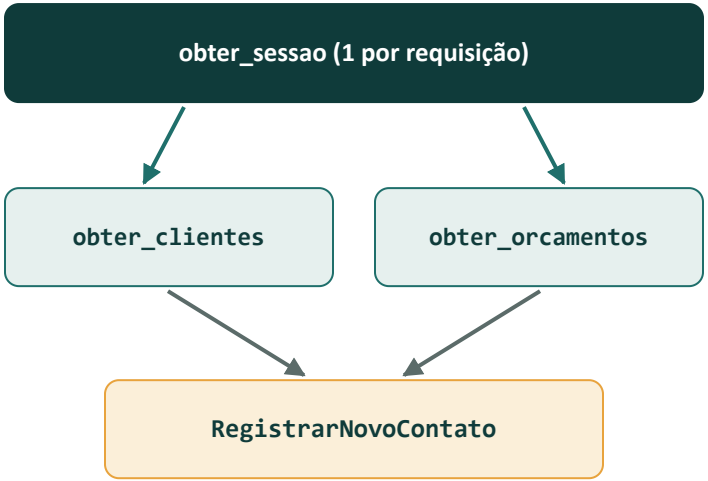
ETAPA 5

Injeção de dependências e uma transação por requisição

```
def obter_sessao(request):
    with sessao_transacional(fabrica) as sessao:
        yield sessao          # a requisição usa aqui

def obter_orcamentos(sessao = Depends(obter_sessao)):
    return OrcamentoRepositoryPostgres(sessao)

Orcamentos = Annotated[OrcamentoRepository,
                       Depends(obter_orcamentos)]
```



Os dois repositórios recebem a mesma sessão: o FastAPI reaproveita o resultado de cada dependência dentro da mesma requisição.

Resultado: Cliente e Orçamento do UC01 entram na mesma transação. Terminou bem, commit; deu erro em qualquer ponto, rollback (Parte 3).

QUANDO ALGO DÁ ERRADO

Um formato único de erro

422 — `ErroDominio`

TransicaoInvalida, FormatoInvalido, ValorInvalido — a regra não deixa

404 — `RecursoNaoEncontrado`

ClienteNaoEncontrado, OrcamentoNaoEncontrado — o id não existe

422 — `RequestValidationError`

DadosInvalidos — JSON fora do contrato (campo faltando, enum desconhecido)

Todos saem assim:

```
{
  "erro": "TransicaoInvalida",
  "mensagem": "Não é possível
     mover de NOVO_CONTATO
     para FECHADO",
  "campo": "estagio"
}
```

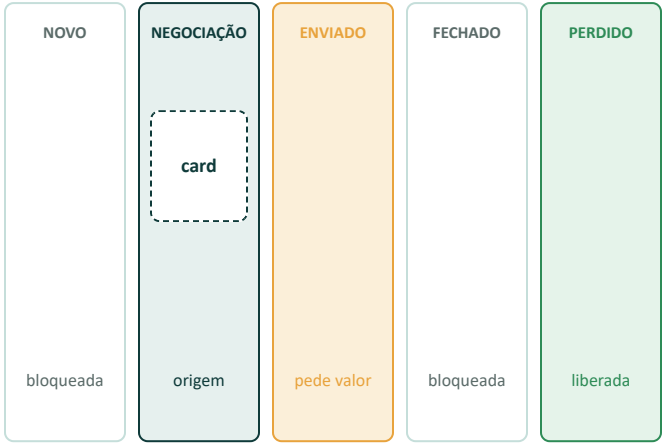
Para que serve no frontend?

- erro diz o tipo do problema — por exemplo, devolver o card à coluna de origem quando vier TransicaoInvalida (RNF02).
- mensagem já vem pronta em português para mostrar ao usuário.
- campo diz qual input destacar no formulário (telefone, email, valor...).

GET /orcamentos/transicoes

```
{
  "transicoes": {
    "NOVO_CONTATO":      ["EM_NEGOCIACAO", "PERDIDO"],
    "EM_NEGOCIACAO":     ["ORCAMENTO_ENVIADO", "PERDIDO"],
    "ORCAMENTO_ENVIADO": ["FECHADO", "PERDIDO"],
    "FECHADO": [],
    "PERDIDO": []
  },
  "exigemValor": ["ORCAMENTO_ENVIADO", "FECHADO"]
}
```

Arrastando um card de EM NEGOCIAÇÃO:



Por que essa rota existe?

O Kanban precisa saber onde o card pode ser solto para dar feedback visual. Esse mapa sai do mesmo enum que o domínio usa: o frontend nunca reimplementa a regra em TypeScript (decisão de arquitetura 08). exigemValor foi acrescentado para avisar antes de soltar.

COMPOSITION ROOT

main.py: onde tudo é ligado

```
def criar_app(configuracao=None, fabrica_sessoes=None,
              relogio=None) -> FastAPI:
    config = configuracao or Configuracao()
    app = FastAPI(title="CRM ...", lifespan=ciclo_de_vida)
    app.state.relogio = relogio or relogio_no_fuso(...)
    app.add_middleware(CORSMiddleware, allow_origins=...)
    registrar_tratadores_de_erro(app)
    for modulo in (cliente_controller, ...):
        app.include_router(modulo.router, prefix="/api/v1")
    return app

app = criar_app()   # usado pelo uvicorn
```

1 **CORS**

Só o endereço do frontend (localhost:3000) pode chamar a API pelo navegador.

2 **Banco só ao subir**

A conexão nasce quando o servidor liga, não quando o arquivo é importado.

3 **Testes**

Passam um SQLite e um relógio de teste para criar_app().

É o único arquivo que conhece todas as camadas. É aqui que se escolhe o “plugue” concreto — o repositório Postgres — para cada interface.

UM BUG QUE O TESTE MANUAL PEGOU

A data do orçamento e o fuso horário

22h30 em Brasília

dia 15/09



= 01h30 em UTC

já é dia 16/09!

✗ Antes

O relógio da aplicação estava em UTC. Um orçamento aberto à noite ganhava a data de amanhã.

```
"dataAbertura": "2026-09-16"
```

✓ Depois

A data do negócio usa FUSO_HORARIO (America/Sao_Paulo). Horários continuam em UTC.

```
"dataAbertura": "2026-09-15"
```

Detalhe do ambiente: o Windows não traz a base de fusos horários que o Python usa, então entrou o pacote tzdata. Um teste fixa 22h30 de Brasília e confere as duas datas.

COMO FOI TESTADO

Testando a API sem abrir o navegador

```
def test_pular_etapa_devolve_422(api, orcamento_id):
    resposta = api.patch(
        f"/api/v1/orcamentos/{orcamento_id}/estagio",
        json={"estagio": "FECHADO"},
    )

    assert resposta.status_code == 422
    assert resposta.json()["erro"] == "TransicaoInvalida"
```

O que os testes cobrem:

- As 9 rotas, nos caminhos de sucesso e de erro
- CORS: libera o frontend e recusa origem estranha
- OpenAPI: todas as rotas aparecem em /docs
- Fuso horário: data local, horário em UTC
- E2E: do primeiro contato ao fechamento

RelogioDeTeste: o tempo também é controlado

Com o relógio parado, duas interações tinham o mesmo horário exato e o histórico saía em ordem trocada — o banco não garante ordem em empate. Na vida real os horários diferem, então o teste agora simula o tempo passando: relogio.avancar(minutos=30).

RESULTADO

Como a Parte 4 terminou

225

testes passando

em cerca de 3 segundos

9

rotas HTTP

todo o contrato, menos login

58

testes pulados

variantes PostgreSQL

1

servidor real testado

uvicorn + curl + /docs

Critério de pronto: pytest verde, mypy --strict e ruff limpos, PATCH de estágio devolvendo 200 na transição válida e 422 na inválida, e um servidor real respondendo.

MÃO NA MASSA

Rode a API e brinque com ela

```
cd backend
set DATABASE_URL=sqlite:///./crm_dev.db

# cria as tabelas (só na primeira vez)
.venv\Scripts\python.exe -m alembic upgrade head

# sobe o servidor
.venv\Scripts\python.exe -m uvicorn main:app --app-dir src --reload
```

Exercício: percorra o funil pelo navegador

1. Abra http://localhost:8000/docs e use POST /clientes (“Try it out”) para criar um contato. Copie o id do orçamento.
2. Em PATCH /orcamentos/{id}/estagio, tente ir direto para FECHADO: veja o 422 TransicaoInvalida.
3. Defina o valor, avance estágio por estágio e confira GET /orcamentos e GET /metricas/funil a cada passo.

PARA REVISAR

Glossário rápido e próximo passo

Rota (endpoint)	Método + endereço: PATCH /orcamentos/{id}/estagio.
Status HTTP	Número que resume o resultado: 200, 201, 404, 422.
Schema	Contrato do JSON de entrada ou saída (Pydantic).
Controller	Função da rota: recebe, chama o caso de uso, responde.
Dependência	Algo que o FastAPI entrega pronto: sessão, repositório.
CORS	Quem, no navegador, pode chamar a API.

PRÓXIMO PASSO

Parte 5 — Autenticação

- POST /auth/login devolvendo um token JWT.
- Todas as rotas passam a exigir Authorization: Bearer.
- Senhas guardadas só como hash.
- Gestão de usuários só para Administrador (RF10).
- Teste de carga: painel com 500 orçamentos em menos de 2 s.